# 作業3：簡易MPI圓形渲染器

圓形渲染輸出範例

## 1. 引言

本次作業實作了一個簡單的基於 MPI 的圓形渲染器。程式讀取二進位場景描述，將圓形柵格化為影像，並輸出 PNG 格式的影像。目標是並行化渲染器（MPI），測量效能，並分析其擴展性和負載平衡。

### 資料模型（每個圓圈）：

- float3 位置 影像平面上的二維位置 (x, y) float 半徑 圓的半徑
- 
- float3 顏色 RGB 顏色

### 演算法（高階層面）：

```
清除每個圓的影像緩衝區（從後
往前順序）：
    計算螢幕空間邊界框，針對邊界框內的每個像素：計算像素
    中心；如果像素中心位於圓內：計算圓的顏色貢獻並混合
        到像素中
```

提供的框架包含二進位格式的讀取器和寫入器；除非另有說明，否則您無需修改檔案 I/O。

注意：二進位場景檔案依繪製順序儲存圓形一條目從後往前排列（檔案順序 = 從後到前）。如果按順序讀取文件，則會從後往前處理圓。

## 二進位檔案格式

渲染器需要緊湊的二進位場景格式（小端序）。字段包括：

- 標題：
  - 4 位元組：魔術 ASCII CRDR 4
  - 位元組：版本（uint32）

    - 8 位元組：計數 (uint64)　圓記錄數 6 個浮點數：邊界框（float32 類型）
    - (xmin, ymin, zmin, xmax, ymax, zmax)

- 每筆記錄的佈局（壓縮，小端序）：
    - float32 x　圓心 X
    - float32 y　圓心 Y 座標；float32 radius
    - 　圓半徑
    - uint8 r, g, b　顏色通道 (0..255)

因此，每筆記錄為 3*4 + 3 = 15 位元組（每筆記錄沒有深度z軸，也沒有 alpha 通道）。文件按從後到前的繪製順序列出圓（較早的記錄位於更靠後的位置）。

筆記：

- 當邊界框表示像素空間時（例如，xmin=0，ymin=0，xmax=W，ymax=H），可以使用頭部邊界框來推斷影像大小。

- 所有數值欄位均為小端位元組序（與提供的生成器和讀取器相符）。

## 2. 要求

1. 實現正確的序列渲染器（單一進程）。
2. 使用 MPI 平行化渲染器
3. 報告並分析結果。

### 關於您的報告的一般提示

- 描述輸入尺寸及其對運行時間的影響（影像大小、圓的數量）。
- 報告運行時間與進程數的關係（強擴展性）。
- 報告每個進程的運行時間與問題規模的關係（弱擴展性）。
- 報告相對於串行基準的速度和效率。
- 你應用了哪些優化技術？
- 你想分享什麼都可以。

### 筆記

- 查看實驗日誌中每個排名對應的時間和負載統計資料。你是否觀察到負載不平衡的情況？如果是，請量化其程度（例如，最大/平均時間、標準差），這或許能幫助你在競爭中脫穎而出。

## 3. 環境

- 作業系統：Ubuntu 24.04 伺服器版
- CPU數量：144
- 插槽：2
- 每個插槽的核心數：36
- 每個核心的線程數：2
- 網路連線：所有節點均透過 Mellanox ConnectX-5 適配器連接至相同交換器。
- 節點總數：8

### 節點分配和限制

為了容納所有學生，班級被分成兩組：

- 第 1 組（1 隊至 25 隊）：節點 0 至節點 3
- 第 2 組（25 隊至 50 隊）：節點 4 至節點 7

限制：為盡量減少干擾，每個團隊在每個節點上建立的運算流程/執行緒不應超過 16 個。

### 連接資訊（SSH）

- MPI節點IP： 172.16.179.50 -- 172.16.179.57
- 跳轉伺服器： 140.112.90.37 （連接埠：9037）
    - 南洋理工大學資訊工程系學生：如果您的客戶端位於172.16.0.0/16子網路內，則可以直接登入 MPI 節點。連線到CSIE VPN。第一的。
    - 其他學生：請先登入跳轉伺服器，然後從那裡存取 MPI 節點。

警告：請勿在跳轉伺服器上執行計算進程！

## 4. 執行

## MPI 設定

> 您應該完成這部分才能運行您的 MPI 程式。

1. 在其中一個 MPI 節點上產生 SSH 金鑰對（密碼短語留空）：

```
ssh-keygen -t ed25519
```

2. 使所有你被允許存取的節點都能辨識你的公鑰：

```
ssh-copy-id -i ~/.ssh/id_ed25519.pub rdmaX
```

rdmaX是172.16.179.4X的主機名，可在 MPI 節點中辨識。請記得將X替換為某個數字。

3. 將私鑰複製到所有你有權存取的節點：

```
scp ~/.ssh/id_ed25519 rdmaX:.ssh
```

> 為了安全起見，建議將私鑰留在 MPI 節點上。如需從外部訪問，請在您的 PC 或跳轉伺服器上產生另一對金鑰，然後執行 `ssh-copy-id`命令將公鑰傳送到 MPI 節點。

### MPI 運行

> 你應該將程式碼和測試資料放在所有 MPI 節點上的相同絕對路徑下。
> 請務必事先將測試資料複製到所有 MPI 節點上。此外，使用 mpicc 編譯程式碼後，請務必將編譯後的程式碼複製到所有 MPI 節點的相同路徑下。
>
> 順便說一下，雖然測試數據放置在所有節點上，但只有 rank 0 進程會讀取數據，並且要求您從 rank 0 進程分發這些數據。

我們將按如下方式運行您的程式碼：

```
# 編譯（連結數學函式庫並啟用 CPU 特定最佳化） mpicc renderer_mpi.c -o renderer_mpi -lm -march=native


# 將二進位檔案複製到其他節點（例如節點 rdma1..rdma3） for i in 1 2 3; do scp renderer_mpi
rdma$i:~/; done

# 在 4 個節點上運行，每個節點 16 個進程UCX_TLS=rc,sm,self \
UCX_NET_DEVICES=mlx5_0:1 \
UCX_LOG_LEVEL=info \ mpirun \ --hostfile
hosts \ -npernode 16 \ --mcatest
p ucx
    \mcatestr.
```

主機內容：

```
rdma0
rdma1
rdma2
rdma3
```

為了進行測試，您可以嘗試不同的主機檔案配置。請注意，不同的團隊可以存取不同的節點組。

## 5. 提交

- 截止日期：4月24日（週五）23:59 提
- 交方式：由團隊成員上傳Team_<團隊編號>_HW3.zip檔 所需檔案：1. renderer.c 序列實作 2.
- renderer_omp.c
  最終最佳化實作 3. Team_<團隊編號
  >_HW3_report.pdf

請確保您的原始程式碼可以順利編譯和運行。如果您的程式碼需要修改才能編譯和運行，將會被扣除少量分數。

# 6. 評分政策

1. 正確率 (10%)
    - 你在本部分獲得的分數與你通過的測試案例數量成正比。
    - 與黃金輸出無差異（diff == 0），正確性無誤。
2. 可擴展性（10%）
    - 只要你的平行化程式碼能展現出良好的可擴展性，你就能獲得滿分。
    - 請在報告中論證您項目的可擴充性。
3. 競賽排名（40%）
    - 分數公式：分數 = max(10, 40 × (100 − 5 × (排名 − 1))%)。
    - 但是，如果你的程式給出錯誤的答案，這部分你將得0分。
4. 報告（40%）
    - 你的分析越透徹、越合理、越清晰，這部分得分就越高。

公平性和編譯標準：為確保所有提交內容的絕對公平性，評分將嚴格使用第 4 節中提供的編譯和執行命令。在正式排名和評分過程中，不會應用除提供的命令之外的任何其他標誌、自訂編譯器或特定於環境的最佳化。

警告：

- 截止日期後提交的作業，其總分將乘以 70%。
- 請遵循模板程式碼說明，不要修改不應更改的程式碼。